

`ApplicationExecutionState`. Its values and the appropriate action to take due to that state are as follows:

 Expand table

ApplicationExecutionState	Explanation	Action to take
NotRunning	An app could be in this state because it hasn't been launched since the last time the user rebooted or logged in. It can also be in this state if it was running but then crashed, or because the user closed it earlier.	Initialize the app as if it is running for the first time in the current user session.
Suspended	The user either minimized or switched away from your app and didn't return to it within a few seconds.	When the app was suspended, its state remained in memory. You only need to reacquire any file handles or other resources you released when the app was suspended.
Terminated	The app was previously suspended but was then shutdown at some point because the system needed to reclaim memory.	Restore the state that the app was in when the user switched away from it.
ClosedByUser	The user closed the app with the system close button, or with Alt+F4. When the user closes the app, it is first suspended and then terminated.	Because the app has essentially gone through the same steps that lead to the Terminated state, handle this the same way you would the Terminated state.
Running	The app was already open when the user tried to launch it again.	Nothing. Note that another instance of your app is not launched. The already running instance is simply activated.

Note

Current user session is based on Windows logon. As long as the current user hasn't logged off, shut down, or restarted Windows, the current user session persists across events such as lock screen authentication, switch-user, and so on.

One important circumstance to be aware of is that if the device has sufficient resources, the operating system will prelaunch frequently used apps that have opted in for that behavior in order to optimize responsiveness. Apps that are prelaunched are launched in the background and then quickly suspended so that when the user switches to them, they can be resumed which is faster than launching the app.

Because of prelaunch, the app's **OnLaunched()** method may be initiated by the system rather than by the user. Because the app is prelaunched in the background you may need to take different action in **OnLaunched()**. For example, if your app starts playing music when launched, they will not know where it is coming from because the app is prelaunched in the background. Once your app is prelaunched in the background, it is followed by a call to **Application.Suspending**. Then, when the user does launch the app, the resuming event is invoked as well as the **OnLaunched()** method. See [Handle app prelaunch](#) for additional information about how to handle the prelaunch scenario. Only apps that opt-in are prelaunched.

Windows displays a splash screen for the app when it is launched. To configure the splash screen, see [Adding a splash screen](#).

While the splash screen is displayed, your app should register event handlers and set up any custom UI it needs for the initial page. See that these tasks running in the application's constructor and **OnLaunched()** are completed within a few seconds or the system may think your app is unresponsive and terminate it. If an app needs to request data from the network or needs to retrieve large amounts of data from disk, these activities should be completed outside of launch. An app can use its own custom loading UI or an extended splash screen while it waits for long running operations to finish. See [Display a splash screen for more time](#) and the [Splash screen sample](#)  for more info.

After the app completes launching, it enters the **Running** state and the splash screen disappears and all splash screen resources and objects are cleared.

App activation

In contrast to being launched by the user, an app can be activated by the system. An app may be activated by a contract such as the share contract. Or it may be activated to handle a custom URI protocol or a file with an extension that your app is registered to handle. For a list of ways your app can be activated, see [ActivationKind](#).

The [Windows.UI.Xaml.Application](#) class defines methods you can override to handle the various ways your app may be activated. **OnActivated** can handle all possible activation types. However, it's more common to use specific methods to handle the

most common activation types, and use **OnActivated** as the fallback method for the less common activation types. These are the additional methods for specific activations:

[OnCachedFileUpdaterActivated](#)

[OnFileActivated](#)

[OnFileOpenPickerActivated](#) [OnFileSavePickerActivated](#)

[OnSearchActivated](#)

[OnShareTargetActivated](#)

The event data for these methods includes the same [PreviousExecutionState](#) property that we saw above, which tells you which state your app was in before it was activated. Interpret the state and what you should do it the same way as described above in the [App launch](#) section.

Note If you log on using the computer's Administrator account, you can't activate UWP apps.

Running in the background

Starting with Windows 10, version 1607, apps can run background tasks within the same process as the app itself. Read more about it in [Background activity with the Single Process Model](#) [↗](#). We won't go into in-process background processing in this article, but how this impacts the app lifecycle is that two new events have been added related to when your app is in the background. They are: [EnteredBackground](#) and [LeavingBackground](#).

These events also reflect whether the user can see your app's UI.

Running in the background is the default state that an application is launched, activated, or resumed into. In this state your application UI is not visible yet.

Running in the foreground

Running in the foreground means that your app's UI is visible.

The **LeavingBackground** event is fired just before your application UI is visible and before entering the running in foreground state. It also fires when the user switches back to your app.

Previously, the best location to load UI assets was in the **Activated** or **Resuming** event handlers. Now **LeavingBackground** is the best place to verify that your UI is ready.

It is important to check that visual assets are ready by this time because this is the last opportunity to do work before your application is visible to the user. All UI work in this event handler should complete quickly, as it impacts the launch and resume time that the user experiences. **LeavingBackground** is the time to ensure the first frame of UI is ready. Then, long-running storage or network calls should be handled asynchronously so that the event handler may return.

When the user switches away from your application, your app reenters the running in background state.

Reentering the background state

The **EnteredBackground** event indicates that your app is no longer visible in the foreground. On the desktop **EnteredBackground** fires when your app is minimized; on phone, when switching to the home screen or another app.

Reduce your app's memory usage

Since your app is no longer visible to the user, this is the best place to stop UI rendering work and animations. You can use **LeavingBackground** to start that work again.

If you are going to do work in the background, this is the place to prepare for it. It is best to check [MemoryManager.AppMemoryUsageLevel](#) and, if needed, reduce the amount of memory being used by your app when it is running in the background so that your app doesn't risk being terminated by the system to free up resources.

See [Reduce memory usage when your app moves to the background state](#) for more details.

Save your state

The suspending event handler is the best place to save your app state. However, if you are doing work in the background (for example, audio playback, using an extended execution session or in-proc background task), it is also a good practice to save your data asynchronously from your **EnteredBackground** event handler. This is because it is possible for your app to be terminated while it is at a lower priority in the background. And because the app will not have gone through the suspended state in that case, your data will be lost.

Saving your data in your **EnteredBackground** event handler, before background activity begins, ensures a good user experience when the user brings your app back to the

foreground. You can use the application data APIs to save data and settings. For more info, see [Store and retrieve settings and other app data](#).

After you save your data, if you are over your memory usage limit, then you can release your data from memory since you can reload it later. That will free memory that can be used by the assets needed for background activity.

Be aware that if your app has background activity in progress that it can move from the running in the background state to the running in the foreground state without ever reaching the suspended state.

⚠ Note

When your app is being closed by the user, it is possible for the **OnSuspending** event to be fired before the **EnteredBackground** event. In some cases, the **EnteredBackground** event may not be fired before the app is terminated. It is important to save your data in the **OnSuspending** event handler.

Asynchronous work and Deferrals

If you make an asynchronous call within your handler, control returns immediately from that asynchronous call. That means that execution can then return from your event handler and your app will move to the next state even though the asynchronous call hasn't completed yet. Use the [GetDeferral](#) method on the [EnteredBackgroundEventArgs](#) object that is passed to your event handler to delay suspension until after you call the [Complete](#) method on the returned [Windows.Foundation.Deferral](#) object.

A deferral doesn't increase the amount of time you have to run your code before your app is terminated. It only delays termination until either the deferral's *Complete* method is called, or the deadline passes-*whichever comes first*.

If you need more time to save your state, investigate ways to save your state in stages before your app enters the background state so that there is less to save in your **OnSuspending** event handler. Or you may request an [ExtendedExecutionSession](#) to get more time. There is no guarantee that the request will be granted, however, so it is best to find ways to minimize the amount of time you need to save your state.

App suspend

When the user minimizes an app Windows waits a few seconds to see whether the user will switch back to it. If they do not switch back within this time window, and no extended execution, background task, or activity sponsored execution is active, Windows suspends the app. An app is also suspended when the lock screen appears as long as no extended execution session, etc. is active in that app.

When an app is suspended, it invokes the [Application.Suspending](#) event. Visual Studio's UWP project templates provide a handler for this event called **OnSuspending** in **App.xaml.cs**. You should put the code to save your application state here.

You should release exclusive resources and file handles so that other apps can access them while your app is suspended. Examples of exclusive resources include cameras, I/O devices, external devices, and network resources. Explicitly releasing exclusive resources and file handles helps to ensure that other apps can access them while your app is suspended. When the app is resumed, it should reacquire its exclusive resources and file handles.

Be aware of the deadline

In order to ensure a fast and responsive device, there is a limit for the amount of time you have to run your code in your suspending event handler. It is different for each device, and you can find out what it is using a property of the [SuspendingOperation](#) object called the deadline.

As with the **EnteredBackground** event handler, if you make an asynchronous call from your handler, control returns immediately from that asynchronous call. That means that execution can then return from your event handler and your app will move to the suspend state even though the asynchronous call hasn't completed yet. Use the [GetDeferral](#) method on the [SuspendingOperation](#) object (available via the event args) to delay entering the suspended state until after you call the [Complete](#) method on the returned [SuspendingDeferral](#) object.

If you need more time, you may request an [ExtendedExecutionSession](#). There is no guarantee that the request will be granted, however, so it is best to find ways to minimize the amount of time you need in your **Suspended** event handler.

App terminate

The system attempts to keep your app and its data in memory while it's suspended. However, if the system does not have the resources to keep your app in memory, it will terminate your app. Apps don't receive a notification that they are being terminated, so

the only opportunity you have to save your app's data is in your **OnSuspending** event handler.

When your app determines that it has been activated after being terminated, it should load the application data that it saved so that the app is in the same state it was in before it was terminated. When the user switches back to a suspended app that has been terminated, the app should restore its application data in its **OnLaunched** method. The system doesn't notify an app when it is terminated, so your app must save its application data and release exclusive resources and file handles before it is suspended, and restore them when the app is activated after termination.

A note about debugging using Visual Studio: Visual Studio prevents Windows from suspending an app that is attached to the debugger. This is to allow the user to view the Visual Studio debug UI while the app is running. When you're debugging an app, you can send it a suspend event using Visual Studio. Make sure the **Debug Location** toolbar is being shown, then click the **Suspend** icon.

App resume

A suspended app is resumed when the user switches to it or when it is the active app when the device comes out of a low power state.

When an app is resumed from the **Suspended** state, it enters the **Running in background** state and the system restores the app where it left off so that it appears to the user as if it has been running all along. No app data stored in memory is lost. Therefore, most apps don't need to restore state when they are resumed though they should reacquire any file or device handles that they released when they were suspended, and restore any state that was explicitly released when the app was suspended.

Your app may be suspended for hours or days. If your app has content or network connections that may have gone stale, these should be refreshed when the app resumes. If an app registered an event handler for the **Application.Resuming** event, it is called when the app is resumed from the **Suspended** state. You can refresh your app content and data in this event handler.

If a suspended app is activated to participate in an app contract or extension, it receives the **Resuming** event first, then the **Activated** event.

If the suspended app was terminated, there is no **Resuming** event and instead **OnLaunched()** is called with an **ApplicationExecutionState** of **Terminated**. Because you saved your state when the app was suspended, you can restore that state during

`OnLaunched()` so that your app appears to the user as it was when they switched away from it.

While an app is suspended, it does not receive any network events that it registered to receive. These network events are not queued--they are simply missed. Therefore, your app should test the network status when it is resumed.

Note Because the [Resuming](#) event is not raised from the UI thread, a dispatcher must be used if the code in your resume handler communicates with your UI. See [Update the UI thread from a background thread](#) [↗](#) for a code example of how to do this.

For general guidelines, see [Guidelines for app suspend and resume](#).

App close

Generally, users don't need to close apps, they can let Windows manage them. However, users can choose to close an app using the close gesture or by pressing Alt+F4 or by using the task switcher on Windows Phone.

There is not an event to indicate that the user closed the app. When an app is closed by the user, it is first suspended to give you an opportunity to save its state. In Windows 8.1 and later, after an app has been closed by the user, the app is removed from the screen and switch list but not explicitly terminated.

Closed-by-user behavior: If your app needs to do something different when it is closed by the user than when it is closed by Windows, you can use the activation event handler to determine whether the app was terminated by the user or by Windows. See the descriptions of **ClosedByUser** and **Terminated** states in the reference for the [ApplicationExecutionState](#) enumeration.

We recommend that apps not close themselves programmatically unless absolutely necessary. For example, if an app detects a memory leak, it can close itself to ensure the security of the user's personal data.

App crash

The system crash experience is designed to get users back to what they were doing as quickly as possible. You shouldn't provide a warning dialog or other notification because that will delay the user.

If your app crashes, stops responding, or generates an exception, a problem report is sent to Microsoft per the user's feedback and diagnostics settings. See [Diagnostics](#),

[feedback, and privacy in Windows](#) for more information. Microsoft provides a subset of the error data in the problem report to you so that you can use it to improve your app. You'll be able to see this data in your app's Quality page in your Dashboard.

When the user activates an app after it crashes, its activation event handler receives an **ApplicationExecutionState** value of **NotRunning**, and should display its initial UI and data. After a crash, don't routinely use the app data you would have used for **Resuming** with **Suspended** because that data could be corrupt; see [Guidelines for app suspend and resume](#).

App removal

When a user deletes your app, the app is removed, along with all its local data. Removing an app doesn't affect the user's data that was stored in common locations such as the Documents or Pictures libraries.

App lifecycle and the Visual Studio project templates

The basic code that is relevant to the app lifecycle is provided in the Visual Studio project templates. The basic app handles launch activation, provides a place for you to restore your app data, and displays the primary UI even before you've added any of your own code. For more info, see [C#, VB, and C++ project templates for apps](#).

Key application lifecycle APIs

- [Windows.ApplicationModel](#) namespace
- [Windows.ApplicationModel.Activation](#) namespace
- [Windows.ApplicationModel.Core](#) namespace
- [Windows.UI.Xaml.Application](#) class (XAML)
- [Windows.UI.Xaml.Window](#) class (XAML)

Related topics

- [ApplicationExecutionState](#)
- [Guidelines for app suspend and resume](#)
- [Handle app prelaunch](#)
- [Handle app activation](#)
- [Handle app suspend](#)

- [Handle app resume](#)
- [Background activity with the Single Process Model](#) 
- [Play media in the Background](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Handle app prelaunch

Article • 10/20/2022

Learn how to handle app prelaunch by overriding the [OnLaunched](#) method, and calling `CoreApplication.EnablePrelaunch`.

Introduction

When available system resources allow, the startup performance of UWP apps on desktop device family devices is improved by proactively launching the user's most frequently used apps in the background. A prelaunched app is put into the suspended state shortly after it is launched. Then, when the user invokes the app, the app is resumed by bringing it from the suspended state to the running state—which is faster than launching the app cold. The user's experience is that the app simply launched very quickly.

Prior to Windows 10, apps didn't automatically take advantage of prelaunch. In Windows 10, version 1511, all Universal Windows Platform (UWP) apps were candidates for being prelaunched. In Windows 10, version 1607, you must opt in to prelaunch behavior by calling `CoreApplication.EnablePrelaunch`, and passing `true`. A good place to put this call is within `OnLaunched`, near the location that the `if (e.PrelaunchActivated == false)` check is made.

Whether an app is prelaunched depends on system resources. If the system is experiencing resource pressure, then apps are not prelaunched.

Some types of apps might need to change their startup behavior in order to work well with prelaunch. For example, an app that plays music when it starts up; a game that assumes that the user is present, and displays elaborate visuals when the app starts up; a messaging app that changes the user's online visibility during startup—all of these can identify when the app was prelaunched, and can change their startup behavior as described in the sections below.

The default templates for XAML Projects (C#, VB, C++) accommodate prelaunch.

Prelaunch and the app lifecycle

After an app is prelaunched, it enters the suspended state. (see [Handle app suspend](#)).

Detect and handle prelaunch

Apps receive the [LaunchActivatedEventArgs.PrelaunchActivated](#) flag during activation. Use this flag to run code that should only run when the user explicitly launches the app, as shown in the following modification to [Application.OnLaunched](#).

C#

```
protected override void OnLaunched(LaunchActivatedEventArgs e)
{
    // CoreApplication.EnablePrelaunch was introduced in Windows 10 version
    1607
    bool canEnablePrelaunch =
    Windows.Foundation.Metadata.ApiInformation.IsMethodPresent("Windows.Applicat
    ionModel.Core.CoreApplication", "EnablePrelaunch");

    // NOTE: Only enable this code if you are targeting a version of Windows
    10 prior to version 1607,
    // and you want to opt out of prelaunch.
    // In Windows 10 version 1511, all UWP apps were candidates for
    prelaunch.
    // Starting in Windows 10 version 1607, the app must opt in to be
    prelaunched.
    //if ( !canEnablePrelaunch && e.PrelaunchActivated == true)
    //{
    //    return;
    //}

    Frame rootFrame = Window.Current.Content as Frame;

    // Do not repeat app initialization when the Window already has content,
    // just ensure that the window is active
    if (rootFrame == null)
    {
        // Create a Frame to act as the navigation context and navigate to
        the first page
        rootFrame = new Frame();

        rootFrame.NavigationFailed += OnNavigationFailed;

        if (e.PreviousExecutionState ==
        ApplicationExecutionState.Terminated)
        {
            //TODO: Load state from previously suspended application
        }

        // Place the frame in the current Window
        Window.Current.Content = rootFrame;
    }

    if (e.PrelaunchActivated == false)
    {
```

```

        // On Windows 10 version 1607 or later, this code signals that this
        app wants to participate in prelaunch
        if (canEnablePrelaunch)
        {
            TryEnablePrelaunch();
        }

        // TODO: This is not a prelaunch activation. Perform operations
which
        // assume that the user explicitly launched the app such as updating
        // the online presence of the user on a social network, updating a
        // what's new feed, etc.

        if (rootFrame.Content == null)
        {
            // When the navigation stack isn't restored navigate to the
first page,
            // configuring the new page by passing required information as a
navigation
            // parameter
            rootFrame.Navigate(typeof(MainPage), e.Arguments);
        }
        // Ensure the current window is active
        Window.Current.Activate();
    }
}

/// <summary>
/// This method should be called only when the caller
/// determines that we're running on a system that
/// supports CoreApplication.EnablePrelaunch.
/// </summary>
private void TryEnablePrelaunch()
{
    Windows.ApplicationModel.Core.CoreApplication.EnablePrelaunch(true);
}

```

Important

The **TryEnablePrelaunch** method in the code example above calls **CoreApplication.EnablePrelaunch**. And **TryEnablePrelaunch** is itself called only when the app is running on a version of Windows that supports **CoreApplication.EnablePrelaunch**. In general, if there's any doubt, then you should use a Windows API *only after determining that it is supported by the platform that your code is running on*. And you can do that by means of the **ApiInformation** class, as shown in the code example above.

There's also code in the example above that you can uncomment if your app needs to opt out of prelaunch when running on Windows 10, version 1511. In version 1511, all

UWP apps were automatically opted in to prelaunch, which might not be appropriate for your app.

Use the VisibilityChanged event

Apps activated by prelaunch are not visible to the user. They become visible when the user switches to them. You may want to delay certain operations until your app's main window becomes visible. For example, if your app displays a list of what's new items from a feed, you could update the list during the **VisibilityChanged** event rather than use the list that was built when the app was prelaunched because it may become stale by the time the user activates the app. The following code handles the **VisibilityChanged** event for **MainPage**:

C#

```
public sealed partial class MainPage : Page
{
    public MainPage()
    {
        this.InitializeComponent();

        Window.Current.VisibilityChanged +=
        WindowVisibilityChangedEventHandler;
    }

    void WindowVisibilityChangedEventHandler(System.Object sender,
    Windows.UI.Core.VisibilityChangedEventArgs e)
    {
        // Perform operations that should take place when the application
        becomes visible rather than
        // when it is prelaunched, such as building a what's new feed
    }
}
```

DirectX games guidance

DirectX games should generally not enable prelaunch because many DirectX games do their initialization before prelaunch can be detected. Starting with Windows 1607, Anniversary edition, your game will not be prelaunched by default. If you do want your game to take advantage of prelaunch, call **CoreApplication.EnablePrelaunch(true)**.

If your game targets an earlier version of Windows 10, you can handle the prelaunch condition to exit the application:

C++/WinRT

```

void ViewProvider::OnActivated(CoreApplicationView const& /* appView */,
Windows::ApplicationModel::Activation::IActivatedEventArgs const& args)
{
    if (args.Kind() ==
Windows::ApplicationModel::Activation::ActivationKind::Launch)
    {
        auto launchArgs{
args.as<Windows::ApplicationModel::Activation::LaunchActivatedEventArgs>());
        if (launchArgs.PrelaunchActivated())
        {
            // Opt-out of Prelaunch.
            CoreApplication::Exit();
        }
    }
}

void ViewProvider::Initialize(CoreApplicationView const & appView)
{
    appView.Activated({ this, &App::OnActivated });
}

```

C++

```

void ViewProvider::OnActivated(CoreApplicationView^
appView,IActivatedEventArgs^ args)
{
    if (args->Kind == ActivationKind::Launch)
    {
        auto launchArgs = static_cast<LaunchActivatedEventArgs^>(args);
        if (launchArgs->PrelaunchActivated)
        {
            // Opt-out of Prelaunch
            CoreApplication::Exit();
            return;
        }
    }
}

```

General guidance

- Apps should not perform long running operations during prelaunch because the app will terminate if it can't be suspended quickly.
- Apps should not initiate audio playback from [Application.OnLaunched](#) when the app is prelaunched because the app won't be visible and it won't be apparent why there is audio playing.
- Apps should not perform any operations during launch which assume that the app is visible to the user, or assume that the app was explicitly launched by the user. Because an app can now be launched in the background without explicit user

action, developers should consider the privacy, user experience and performance implications.

- An example privacy consideration is when a social app should change the user state to online. It should wait until the user switches to the app instead of changing the status when the app is prelaunched.
- An example user experience consideration is that if you have an app, such as a game, that displays an introductory sequence when it is launched, you might delay the introductory sequence until the user switches to the app.
- An example performance implication is that you might wait until the user switches to the app to retrieve the current weather information instead of loading it when the app is prelaunched and then need to load it again when the app becomes visible to ensure that the information is current.
- If your app clears its Live Tile when launched, defer doing this until the visibility changed event.
- Telemetry for your app should distinguish between normal tile activations and prelaunch activations to make it easier to narrow down the scenario if problems occur.
- If you have Microsoft Visual Studio 2015 Update 1, and Windows 10, Version 1511, then you can simulate prelaunch for App your app in Visual Studio 2015 by choosing **Debug > Other Debug Targets > Debug Windows Universal App PreLaunch**.

Related topics

- [App lifecycle](#)
- [CoreApplication.EnablePrelaunch](#)

Handle app activation

Article • 10/20/2022

Learn how to handle app activation by overriding the [Application.OnLaunched](#) method.

ⓘ Note

For info about handling activation in a desktop app, see [Get activation info for packaged apps](#). Also see [AppLifecycle—rich activation](#) [↗] on GitHub.

Override the launch handler

When an app is activated, for any reason, the system sends the [CoreApplicationView.Activated](#) event. For a list of activation types, see the [ActivationKind](#) enumeration.

The [Windows.UI.Xaml.Application](#) class defines methods you can override to handle the various activation types. Several of the activation types have a specific method that you can override. For the other activation types, override the [OnActivated](#) method.

Define the class for your application.

XML

```
<Application
  x:Class="AppName.App"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
```

Override the [OnLaunched](#) method. This method is called whenever the user launches the app. The [LaunchActivatedEventArgs](#) parameter contains the previous state of your app and the activation arguments.

ⓘ Note

On Windows, launching a suspended app from Start tile or app list doesn't call this method.

C#

```

using System;
using Windows.ApplicationModel.Activation;
using Windows.UI.Xaml;

namespace AppName
{
    public partial class App
    {
        async protected override void OnLaunched(LaunchActivatedEventArgs
args)
        {
            EnsurePageCreatedAndActivate();

            // Creates the MainPage if it isn't already created. Also activates
            // the window so it takes foreground and input focus.
            private MainPage EnsurePageCreatedAndActivate()
            {
                if (Window.Current.Content == null)
                {
                    Window.Current.Content = new MainPage();
                }

                Window.Current.Activate();
                return Window.Current.Content as MainPage;
            }
        }
    }
}

```

Restore application data if app was suspended then terminated

When the user switches to your terminated app, the system sends the **Activated** event, with **Kind** set to **Launch** and **PreviousExecutionState** set to **Terminated** or **ClosedByUser**. The app should load its saved application data and refresh its displayed content.

C#

```

async protected override void OnLaunched(LaunchActivatedEventArgs args)
{
    if (args.PreviousExecutionState == ApplicationExecutionState.Terminated
||
    args.PreviousExecutionState ==
ApplicationExecutionState.ClosedByUser)
    {
        // TODO: Populate the UI with the previously saved application data
    }
    else

```

```
{  
    // TODO: Populate the UI with defaults  
}  
  
EnsurePageCreatedAndActivate();  
}
```

If the value of [PreviousExecutionState](#) is **NotRunning**, the app failed to save its application data successfully and the app should start over as if it were being initially launched.

Remarks

ⓘ Note

Apps can skip initialization if there is already content set on the current window. You can check the [LaunchActivatedEventArgs.TileId](#) property to determine whether the app was launched from a primary or a secondary tile and, based on that information, decide whether you should present a fresh or resume app experience.

Important APIs

- [Windows.ApplicationModel.Activation](#)
- [Windows.UI.Xaml.Application](#)

Related topics

- [Handle app suspend](#)
- [Handle app resume](#)
- [Guidelines for app suspend and resume](#)
- [App lifecycle](#)

Handle app suspend

Article • 03/22/2023

Important APIs

- [Suspending](#)

Learn how to save important application data when the system suspends your app. The example registers an event handler for the [Suspending](#) event and saves a string to a file.

Register the suspending event handler

Register to handle the [Suspending](#) event, which indicates that your app should save its application data before the system suspends it.

C#

```
using System;
using Windows.ApplicationModel;
using Windows.ApplicationModel.Activation;
using Windows.UI.Xaml;

partial class MainPage
{
    public MainPage()
    {
        InitializeComponent();
        Application.Current.Suspending += new
        SuspendingEventHandler(App_Suspending);
    }
}
```

Save application data before suspension

When your app handles the [Suspending](#) event, it has the opportunity to save its important application data in the handler function. The app should use the [LocalSettings](#) storage API to save simple application data synchronously.

C#

```
partial class MainPage
{
    async void App_Suspending(
        Object sender,
        Windows.ApplicationModel.SuspendingEventArgs e)
```

```
{  
    // TODO: This is the time to save app data in case the process is  
    terminated.  
}  
}
```

Release resources

You should release exclusive resources and file handles so that other apps can access them while your app is suspended. Examples of exclusive resources include cameras, I/O devices, external devices, and network resources. Explicitly releasing exclusive resources and file handles helps to ensure that other apps can access them while your app is suspended. When the app is resumed, it should reacquire its exclusive resources and file handles.

Remarks

The system suspends your app whenever the user switches to another app or to the desktop or Start screen. The system resumes your app whenever the user switches back to it. When the system resumes your app, the content of your variables and data structures is the same as it was before the system suspended the app. The system restores the app exactly where it left off, so that it appears to the user as if it's been running in the background.

The system attempts to keep your app and its data in memory while it's suspended. However, if the system does not have the resources to keep your app in memory, the system will terminate your app. When the user switches back to a suspended app that has been terminated, the system sends an [Activated](#) event and should restore its application data in its [OnLaunched](#) method.

The system doesn't notify an app when it's terminated, so your app must save its application data and release exclusive resources and file handles when it's suspended, and restore them when the app is activated after termination.

If you make an asynchronous call within your handler, control returns immediately from that asynchronous call. That means that execution can then return from your event handler and your app will move to the next state even though the asynchronous call hasn't completed yet. Use the [GetDeferral](#) method on the [EnteredBackgroundEventArgs](#) object that is passed to your event handler to delay suspension until after you call the [Complete](#) method on the returned [Windows.Foundation.Deferral](#) object.

A deferral doesn't increase the amount of time you have to run your code before your app is terminated. It only delays termination until either the deferral's *Complete* method is called, or the deadline passes-*whichever comes first*. To extend time in the Suspending state use [ExtendedExecutionSession](#)

ⓘ Note

To improve system responsiveness in Windows 8.1, apps are given low priority access to resources after they are suspended. To support this new priority, the suspend operation timeout is extended so that the app has the equivalent of the 5-second timeout for normal priority on Windows or between 1 and 10 seconds on Windows Phone. You cannot extend or alter this timeout window.

A note about debugging using Visual Studio: Visual Studio prevents Windows from suspending an app that is attached to the debugger. This is to allow the user to view the Visual Studio debug UI while the app is running. When you're debugging an app, you can send it a suspend event using Visual Studio. Make sure the **Debug Location** toolbar is being shown, then click the **Suspend** icon.

Related topics

- [App lifecycle](#)
- [Handle app activation](#)
- [Handle app resume](#)
- [UX guidelines for launch, suspend, and resume](#)
- [Extended Execution](#)

Handle app resume

Article • 03/22/2023

Important APIs

- [Resuming](#)

Learn where to refresh your UI when the system resumes your app. The example in this topic registers an event handler for the [Resuming](#) event.

Register the resuming event handler

Register to handle the [Resuming](#) event, which indicates that the user switched away from your app and then back to it.

C#

```
partial class MainPage
{
    public MainPage()
    {
        InitializeComponent();
        Application.Current.Resuming += new EventHandler<Object>
(App_Resuming);
    }
}
```

Refresh displayed content and reacquire resources

The system suspends your app a few seconds after the user switches to another app or to the desktop. The system resumes your app when the user switches back to it. When the system resumes your app, the content of your variables and data structures are the same as they were before the system suspended the app. The system restores the app where it left off. To the user, it appears as if the app has been running in the background.

When your app handles the [Resuming](#) event, your app may have been suspended for hours or days. It should refresh any content that might have become stale while the app was suspended, such as news feeds or the user's location.

This is also a good time to restore any exclusive resources that you released when your app was suspended such as file handles, cameras, I/O devices, external devices, and network resources.

C#

```
partial class MainPage
{
    private void App_Resuming(Object sender, Object e)
    {
        // TODO: Refresh network data, perform UI updates, and reacquire
        resources like cameras, I/O devices, etc.
    }
}
```

ⓘ Note

Because the **Resuming** event is not raised from the UI thread, a dispatcher must be used in your handler to dispatch any calls to your UI.

Remarks

When your app is attached to the Visual Studio debugger, it will not be suspended. You can suspend it from the debugger, however, and then send it a **Resume** event so that you can debug your code. Make sure the **Debug Location toolbar** is visible and click the drop-down next to the **Suspend** icon. Then choose **Resume**.

For Windows Phone Store apps, the **Resuming** event is always followed by **OnLaunched**, even when your app is currently suspended and the user re-launches your app from a primary tile or app list. Apps can skip initialization if there is already content set on the current window. You can check the **LaunchActivatedEventArgs.TileId** property to determine if the app was launched from a primary or a secondary tile and, based on that information, decide whether you should present a fresh or resume app experience.

Related topics

- [App lifecycle](#)
- [Handle app activation](#)
- [Handle app suspend](#)

Free memory when your app moves to the background

Article • 10/20/2022

This article shows you how to reduce the amount of memory that your app uses when it moves to the background state so that it won't be suspended and possibly terminated.

New background events

Windows 10, version 1607, introduces two new application lifecycle events, [EnteredBackground](#) and [LeavingBackground](#). These events let your app know when it is entering and leaving the background.

When your app moves into the background, the memory constraints enforced by the system may change. Use these events to check your current memory consumption and free resources in order to stay below the limit so that your app won't be suspended and possibly terminated while it is in the background.

Events for controlling your app's memory usage

[MemoryManager.AppMemoryUsageLimitChanging](#) is raised just before the limit of total memory the app can use is changed. For example, when the app moves into the background and on the Xbox the memory limit changes from 1024MB to 128MB. This is the most important event to handle to keep the platform from suspending or terminating the app.

[MemoryManager.AppMemoryUsageIncreased](#) is raised when the app's memory consumption has increased to a higher value in the [AppMemoryUsageLevel](#) enumeration. For example, from **Low** to **Medium**. Handling this event is optional but recommended because the application is still responsible for staying under the limit.

[MemoryManager.AppMemoryUsageDecreased](#) is raised when the app's memory consumption has decreased to a lower value in the [AppMemoryUsageLevel](#) enumeration. For example, from **High** to **Low**. Handling this event is optional but indicates the application may be able to allocate additional memory if needed.

Handle the transition between foreground and background

When your app moves from the foreground to the background, the [EnteredBackground](#) event is raised. When your app returns to the foreground, the [LeavingBackground](#) event is raised. You can register handlers for these events when your app is created. In the default project template, this is done in the **App** class constructor in App.xaml.cs.

Because running in the background will reduce the memory resources your app is allowed to retain, you should also register for the [AppMemoryUsageIncreased](#) and [AppMemoryUsageLimitChanging](#) events which you can use to check your app's current memory usage and the current limit. The handlers for these events are shown in the following examples. For more information on the application lifecycle for UWP apps, see [App lifecycle](#).

C#

```
public App()
{
    this.InitializeComponent();

    this.Suspending += OnSuspending;

    // Subscribe to key lifecycle events to know when the app
    // transitions to and from foreground and background.
    // Leaving the background is an important transition
    // because the app may need to restore UI.
    this.EnteredBackground += AppEnteredBackground;
    this.LeavingBackground += AppLeavingBackground;

    // During the transition from foreground to background the
    // memory limit allowed for the application changes. The application
    // has a short time to respond by bringing its memory usage
    // under the new limit.
    Windows.System.MemoryManager.AppMemoryUsageLimitChanging +=
MemoryManager_AppMemoryUsageLimitChanging;

    // After an application is backgrounded it is expected to stay
    // under a memory target to maintain priority to keep running.
    // Subscribe to the event that informs the app of this change.
    Windows.System.MemoryManager.AppMemoryUsageIncreased +=
MemoryManager_AppMemoryUsageIncreased;
}
```

When the [EnteredBackground](#) event is raised, set the tracking variable to indicate that you are currently running in the background. This will be useful when you write the code for reducing memory usage.

C#

```
/// <summary>
/// The application entered the background.
```

```

/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
private void AppEnteredBackground(object sender, EnteredBackgroundEventArgs
e)
{
    _isInBackgroundMode = true;

    // An application may wish to release views and view data
    // here since the UI is no longer visible.
    //
    // As a performance optimization, here we note instead that
    // the app has entered background mode with _isInBackgroundMode and
    // defer unloading views until AppMemoryUsageLimitChanging or
    // AppMemoryUsageIncreased is raised with an indication that
    // the application is under memory pressure.
}

```

When your app transitions to the background, the system reduces the memory limit for the app to ensure that the current foreground app has sufficient resources to provide a responsive user experience

The [AppMemoryUsageLimitChanging](#) event handler lets your app know that its allotted memory has been reduced and provides the new limit in the event arguments passed into the handler. Compare the [MemoryManager.AppMemoryUsage](#) property, which provides your app's current usage, to the [NewLimit](#) property of the event arguments, which specifies the new limit. If your memory usage exceeds the limit, you need to reduce your memory usage.

In this example, this is done in the helper method **ReduceMemoryUsage**, which is defined later in this article.

C#

```

/// <summary>
/// Raised when the memory limit for the app is changing, such as when the
app
/// enters the background.
/// </summary>
/// <remarks>
/// If the app is using more than the new limit, it must reduce memory
within 2 seconds
/// on some platforms in order to avoid being suspended or terminated.
///
/// While some platforms will allow the application
/// to continue running over the limit, reducing usage in the time
/// allotted will enable the best experience across the broadest range of
devices.
/// </remarks>
/// <param name="sender"></param>

```

```

/// <param name="e"></param>
private void MemoryManager_AppMemoryUsageLimitChanging(object sender,
AppMemoryUsageLimitChangingEventArgs e)
{
    // If app memory usage is over the limit, reduce usage within 2 seconds
    // so that the system does not suspend the app
    if (MemoryManager.AppMemoryUsage >= e.NewLimit)
    {
        ReduceMemoryUsage(e.NewLimit);
    }
}

```

📌 Note

Some device configurations will allow an application to continue running over the new memory limit until the system experiences resource pressure, and some will not. On Xbox, in particular, apps will be suspended or terminated if they do not reduce memory to under the new limits within 2 seconds. This means that you can deliver the best experience across the broadest range of devices by using this event to reduce resource usage below the limit within 2 seconds of the event being raised.

It is possible that although your app's memory usage is currently under the memory limit for background apps when it first transitions to the background, it may increase its memory consumption over time and begin to approach the limit. The handler the [AppMemoryUsageIncreased](#) provides an opportunity to check your current usage when it increases and, if necessary, free memory.

Check to see if the [AppMemoryUsageLevel](#) is **High** or **OverLimit**, and if so, reduce your memory usage. In this example this is handled by the helper method, **ReduceMemoryUsage**. You can also subscribe to the [AppMemoryUsageDecreased](#) event, check to see if your app is under the limit, and if so then you know you can allocate additional resources.

C#

```

/// <summary>
/// Handle system notifications that the app has increased its
/// memory usage level compared to its current target.
/// </summary>
/// <remarks>
/// The app may have increased its usage or the app may have moved
/// to the background and the system lowered the target for the app
/// In either case, if the application wants to maintain its priority
/// to avoid being suspended before other apps, it may need to reduce
/// its memory usage.

```

```

///
/// This is not a replacement for handling AppMemoryUsageLimitChanging
/// which is critical to ensure the app immediately gets below the new
/// limit. However, once the app is allowed to continue running and
/// policy is applied, some apps may wish to continue monitoring
/// usage to ensure they remain below the limit.
/// </remarks>
/// <param name="sender"></param>
/// <param name="e"></param>
private void MemoryManager_AppMemoryUsageIncreased(object sender, object e)
{
    // Obtain the current usage level
    var level = MemoryManager.AppMemoryUsageLevel;

    // Check the usage level to determine whether reducing memory is
    // necessary.
    // Memory usage may have been fine when initially entering the
    // background but
    // the app may have increased its memory usage since then and will need
    // to trim back.
    if (level == AppMemoryUsageLevel.OverLimit || level ==
AppMemoryUsageLevel.High)
    {
        ReduceMemoryUsage(MemoryManager.AppMemoryUsageLimit);
    }
}

```

ReduceMemoryUsage is a helper method that you can implement to release memory when your app is over the usage limit while running in the background. How you release memory depends on the specifics of your app, but one recommended way to free up memory is to dispose of your UI and the other resources associated with your app view. To do so, ensure that you are running in the background state then set the **Content** property of your app's window to `null` and unregister your UI event handlers and remove any other references you may have to the page. Failing to unregister your UI event handlers and clearing any other references you may have to the page will prevent the page resources from being released. Then call **GC.Collect** to reclaim the freed up memory immediately. Typically you don't force garbage collection because the system will take care of it for you. In this specific case, we are reducing the amount of memory charged to this application as it goes into the background to reduce the likelihood that the system will determine that it should terminate the app to reclaim memory.

C#

```

/// <summary>
/// Reduces application memory usage.
/// </summary>
/// <remarks>
/// When the app enters the background, receives a memory limit changing
/// event, or receives a memory usage increased event, it can

```

```

/// can optionally unload cached data or even its view content in
/// order to reduce memory usage and the chance of being suspended.
///
/// This must be called from multiple event handlers because an application
may already
/// be in a high memory usage state when entering the background, or it
/// may be in a low memory usage state with no need to unload resources yet
/// and only enter a higher state later.
/// </remarks>
public void ReduceMemoryUsage(ulong limit)
{
    // If the app has caches or other memory it can free, it should do so
now.
    // << App can release memory here >>

    // Additionally, if the application is currently
    // in background mode and still has a view with content
    // then the view can be released to save memory and
    // can be recreated again later when leaving the background.
    if (isInBackgroundMode && Window.Current.Content != null)
    {
        // Some apps may wish to use this helper to explicitly disconnect
        // child references.
        //
        VisualTreeHelper.DisconnectChildrenRecursive(Window.Current.Content);

        // Clear the view content. Note that views should rely on
        // events like Page.Unloaded to further release resources.
        // Release event handlers in views since references can
        // prevent objects from being collected.
        Window.Current.Content = null;
    }

    // Run the GC to collect released resources.
    GC.Collect();
}

```

When the window content is collected, each Frame begins its disconnection process. If there are Pages in the visual object tree under the window content, these will begin firing their Unloaded event. Pages cannot be completely cleared from memory unless all references to them are removed. In the Unloaded callback, do the following to ensure that memory is quickly freed:

- Clear and set any large data structures in your Page to `null`.
- Unregister all event handlers that have callback methods within the Page. Make sure to register those callbacks during the Loaded event handler for the Page. The Loaded event is raised when the UI has been reconstituted and the Page has been added to the visual object tree.
- Call `GC.Collect` at the end of the Unloaded callback to quickly garbage collect any of the large data structures you have just set to `null`. Again, typically you don't

force garbage collection because the system will take care of it for you. In this specific case, we are reducing the amount of memory charged to this application as it goes into the background to reduce the likelihood that the system will determine that it should terminate the app to reclaim memory.

C#

```
private void MainPage_Unloaded(object sender, RoutedEventArgs e)
{
    // << free large data structures and set them to null, here >>

    // Disconnect event handlers for this page so that the garbage
    // collector can free memory associated with the page
    Window.Current.Activated -= Current_Activated;
    GC.Collect();
}
```

In the **LeavingBackground** event handler, set the tracking variable (`isInBackgroundMode`) to indicate that your app is no longer running in the background. Next, check to see if the **Content** of the current window is `null` -- which it will be if you disposed of your app views in order to clear up memory while you were running in the background. If the window content is `null`, rebuild your app view. In this example, the window content is created in the helper method **CreateRootFrame**.

C#

```
/// <summary>
/// The application is leaving the background.
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
private void AppLeavingBackground(object sender, LeavingBackgroundEventArgs e)
{
    // Mark the transition out of the background state
    _isInBackgroundMode = false;

    // Restore view content if it was previously unloaded
    if (Window.Current.Content == null)
    {
        CreateRootFrame(ApplicationExecutionState.Running, string.Empty);
    }
}
```

The **CreateRootFrame** helper method recreates the view content for your app. The code in this method is nearly identical to the **OnLaunched** handler code provided in the default project template. The one difference is that the **Launching** handler determines

the previous execution state from the [PreviousExecutionState](#) property of the [LaunchActivatedEventArgs](#) and the `CreateRootFrame` method simply gets the previous execution state passed in as an argument. To minimize duplicated code, you can refactor the default **Launching** event handler code to call `CreateRootFrame`.

C#

```
void CreateRootFrame(ApplicationExecutionState previousExecutionState,
string arguments)
{
    Frame rootFrame = Window.Current.Content as Frame;

    // Do not repeat app initialization when the Window already has content,
    // just ensure that the window is active
    if (rootFrame == null)
    {
        // Create a Frame to act as the navigation context and navigate to
        the first page
        rootFrame = new Frame();

        // Set the default language
        rootFrame.Language =
Windows.Globalization.ApplicationLanguages.Languages[0];

        rootFrame.NavigationFailed += OnNavigationFailed;

        if (previousExecutionState == ApplicationExecutionState.Terminated)
        {
            //TODO: Load state from previously suspended application
        }

        // Place the frame in the current Window
        Window.Current.Content = rootFrame;
    }

    if (rootFrame.Content == null)
    {
        // When the navigation stack isn't restored navigate to the first
        page,
        // configuring the new page by passing required information as a
        navigation
        // parameter
        rootFrame.Navigate(typeof(MainPage), arguments);
    }
}
```

Guidelines

Moving from the foreground to the background

When an app moves from the foreground to the background, the system does work on behalf of the app to free up resources that are not needed in the background. For example, the UI frameworks flush cached textures and the video subsystem frees memory allocated on behalf of the app. However, an app will still need to carefully monitor its memory usage to avoid being suspended or terminated by the system.

When an app moves from the foreground to the background it will first get an **EnteredBackground** event and then a **AppMemoryUsageLimitChanging** event.

- Do use the **EnteredBackground** event to free up UI resources that you know your app does not need while running in the background. For example, you could free the cover art image for a song.
- Do use the **AppMemoryUsageLimitChanging** event to ensure that your app is using less memory than the new background limit. Make sure that you free up resources if not. If you do not, your app may be suspended or terminated according to device specific policy.
- Do manually invoke the garbage collector if your app is over the new memory limit when the **AppMemoryUsageLimitChanging** event is raised.
- Do use the **AppMemoryUsageIncreased** event to continue to monitor your app's memory usage while running in the background if you expect it to change. If the **AppMemoryUsageLevel** is **High** or **OverLimit** make sure that you free up resources.
- Consider freeing UI resources in the **AppMemoryUsageLimitChanging** event handler instead of in the **EnteredBackground** handler as a performance optimization. Use a boolean value set in the **EnteredBackground/LeavingBackground** event handlers to track whether the app is in the background or foreground. Then in the **AppMemoryUsageLimitChanging** event handler, if **AppMemoryUsage** is over the limit and the app is in the background (based on the Boolean value) you can free UI resources.
- Do not perform long running operations in the **EnteredBackground** event because you can cause the transition between applications to appear slow to the user.

Moving from the background to the foreground

When an app moves from the background to the foreground, the app will first get an **AppMemoryUsageLimitChanging** event and then a **LeavingBackground** event.

- Do use the **LeavingBackground** event to recreate UI resources that your app discarded when moving into the background.

Related topics

- [Background media playback sample](#)  - shows how to free memory when your app moves to the background state.
- [Diagnostic Tools](#)  - use the diagnostic tools to observe garbage collection events and validate that your app is releasing memory the way you expect it to.

Postpone app suspension with extended execution

Article • 10/20/2022

This article shows you how to use extended execution to postpone when your app is suspended so that it can run while minimized or under the lock screen.

When the user minimizes or switches away from an app it is put into a suspended state. Its memory is maintained, but its code does not run. This is true across all OS Editions with a visual user interface. For more details about when your app is suspended, see [Application Lifecycle](#).

There are cases where an app may need to keep running, rather than be suspended, when the user navigates away from the app, or while it is minimized. For example, a step counting app needs to keep running and tracking steps even when the user navigates away to use other apps.

If an app needs to keep running, either the OS can keep it running, or it can request to keep running. For example, when playing audio in the background, the OS can keep an app running longer if you follow these steps for [Background Media Playback](#). Otherwise, you must manually request more time. The amount of time you may get to perform background execution may be several minutes but you must be prepared to handle the session being revoked at any time. These application lifecycle time constraints are disabled while the app is running under a debugger. For this reason it is important to test Extended Execution and other tools for postponing app suspension while not running under a debugger or by using the Lifecycle Events available in Visual Studio.

Create an [ExtendedExecutionSession](#) to request more time to complete an operation in the background. The kind of **ExtendedExecutionSession** you create is determined by the [ExtendedExecutionReason](#) that you provide when you create it. There are three **ExtendedExecutionReason** enum values: **Unspecified**, **LocationTracking** and **SavingData**. Only one **ExtendedExecutionSession** can be requested at any time; attempting to create another session while an approved session request is currently active will cause exception 0x8007139F to be thrown from the **ExtendedExecutionSession** constructor stating that the group or resource is not in the correct state to perform the requested operation. Do not use [ExtendedExecutionForegroundSession](#) and [ExtendedExecutionForegroundReason](#); they require restricted capabilities and are not available for use in Store applications.

Run while minimized

There are two cases where extended execution can be used:

- At any point during regular foreground execution, while the application is in the running state.
- After the application has received a suspending event (the OS is about to move the app to the suspended state) in the application's suspending event handler.

The code for these two cases is the same, but the application behaves a little differently in each. In the first case, the application stays in the running state, even if an event that normally would trigger suspension occurs (for example, the user navigating away from the application). The application will never receive a suspending event while the execution extension is in effect. When the extension is disposed, the application becomes eligible for suspension again.

In the second case, if the application is transitioning to the suspended state, it will stay in a suspending state for the period of the extension. Once the extension expires, the application enters the suspended state without further notification.

Use **ExtendedExecutionReason.Unspecified** when you create an **ExtendedExecutionSession** to request additional time before your app moves into the background for scenarios such as media processing, project compilation, or keeping a network connection alive. On desktop devices running Windows 10 for desktop editions (Home, Pro, Enterprise, and Education), this is the approach to use if an app needs to avoid being suspended while it is minimized.

Request the extension when starting a long running operation in order to defer the **Suspending** state transition that otherwise occurs when the app moves into the background. On desktop devices, extended execution sessions created with **ExtendedExecutionReason.Unspecified** have a battery-aware time limit. If the device is connected to wall power, there is no limit to the length of the extended execution time period. If the device is on battery power, the extended execution time period can run up to ten minutes in the background.

A tablet or laptop user can get the same long running behavior--at the expense of battery life--when the **Allow the app to run background tasks** option is selected in **Battery usage by app** settings. (To find this option on a laptop, go to **Settings > System > Battery > Battery usage by App** (the link under the percent of battery power remaining) > select an app > turn off **Managed By Windows** > select **Allow app to run background tasks**).

On all OS editions this kind of extended execution session stops when the device enters Connected Standby. On mobile devices running Windows 10 Mobile, this kind of extended execution session will run as long as the screen is on. When the screen turns

off, the device immediately attempts to enter the low-power Connected-Standby mode. On desktop devices, the session will continue running if the lock screen appears. The device does not enter Connected Standby for a period of time after the screen turns off. On the Xbox OS Edition, the device enters Connect Standby after one hour unless the user changes the default.

Track the user's location

Specify **ExtendedExecutionReason.LocationTracking** when you create an **ExtendedExecutionSession** if your app needs to regularly log the location from the [GeoLocator](#). Apps for fitness tracking and navigation that need to regularly monitor the user's location and should use this reason.

A location tracking extended execution session can run as long as needed, including while the screen is locked on a mobile device. However, there can only be one such session running per device. A location tracking extended execution session can only be requested in the foreground, and the app must be in the **Running** state. This ensures that the user is aware that the app has initiated an extended location tracking session. It is still possible to use the GeoLocator while the app is in the background by using a background task, or an app service, without requesting a location tracking extended execution session.

Save Critical Data Locally

Specify **ExtendedExecutionReason.SavingData** when you create an **ExtendedExecutionSession** to save user data in the case where not saving the data before the app is terminated will result in data loss and a negative user experience.

Don't use this kind of session to extend the lifetime of an app to upload or download data. If you need to upload data, request a [background transfer](#) or register a **MaintenanceTrigger** to handle the transfer when AC power is available. An **ExtendedExecutionReason.SavingData** extended execution session can be requested either when the app is in the foreground and in the **Running** state, or in the background and in the **Suspending** state.

The **Suspending** state is the last opportunity during the app lifecycle that an app can do work before the app is terminated. **ExtendedExecutionReason.SavingData** is the only type of **ExtendedExecutionSession** that can be requested in the **Suspending** state. Requesting an **ExtendedExecutionReason.SavingData** extended execution session while the app is in the **Suspending** state creates a potential issue that you should be aware of. If an extended execution session is requested while in the **Suspending** state, and the

user requests the app be launched again, it may appear to take a long time to launch. This is because the extended execution session time period must complete before the old instance of the app can be closed and a new instance of the app can be launched. Launch performance time is sacrificed in order to guarantee that user state is not lost.

Request, disposal, and revocation

There are three fundamental interactions with an extended execution session: the request, disposal, and revocation. Making the request is modeled in the following code snippet.

Request

C#

```
var newSession = new ExtendedExecutionSession();
newSession.Reason = ExtendedExecutionReason.Unspecified;
newSession.Revoked += SessionRevoked;
ExtendedExecutionResult result = await newSession.RequestExtensionAsync();

switch (result)
{
    case ExtendedExecutionResult.Allowed:
        DoLongRunningWork();
        break;

    default:
        case ExtendedExecutionResult.Denied:
            DoShortRunningWork();
            break;
}
```

[See code sample](#) 

Calling **RequestExtensionAsync** checks with the operating system to see if the user has approved background activity for the app and whether the system has the available resources to enable background execution. Only one session will be approved for an app at any time, causing additional calls to **RequestExtensionAsync** to result in the session being denied.

You can check the [BackgroundExecutionManager](#) beforehand to determine the [BackgroundAccessStatus](#), which is the user setting that indicates whether your app can run in the background or not. To learn more about these user settings see [Background Activity and Energy Awareness](#) .

The **ExtendedExecutionReason** indicates the operation your app is performing in the background. The **Description** string is a human-readable string that explains why your app needs to perform the operation. This string is not presented to the user, but may be made available in a future release of Windows. The **Revoked** event handler is required so that an extended execution session can halt gracefully if the user, or the system, decides that the app can no longer run in the background.

Revoked

If an app has an active extended execution session and the system requires background activity to halt because a foreground application requires the resources, then the session is revoked. An extended execution session time period is never terminated without first firing the **Revoked** event handler.

When the **Revoked** event is fired for an **ExtendedExecutionReason.SavingData** extended execution session, the app has one second to complete the operation it was performing and finish **Suspending**.

Revocation can occur for many reasons: an execution time limit was reached, a background energy quota was reached, or memory needs to be reclaimed in order for the user to open a new app in the foreground.

Here is an example of a Revoked event handler:

CS

```
private async void SessionRevoked(object sender,
ExtendedExecutionRevokedEventArgs args)
{
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        switch (args.Reason)
        {
            case ExtendedExecutionRevokedReason.Resumed:
                rootPage.NotifyUser("Extended execution revoked due to
returning to foreground.", NotifyType.StatusMessage);
                break;

            case ExtendedExecutionRevokedReason.SystemPolicy:
                rootPage.NotifyUser("Extended execution revoked due to
system policy.", NotifyType.StatusMessage);
                break;
        }

        EndExtendedExecution();
    });
}
```

[See code sample](#)

Dispose

The final step is to dispose of the extended execution session. You want to dispose of the session, and any other memory intensive assets, because otherwise the energy used by the app while it is waiting for the session to close will be counted against the app's energy quota. To preserve as much of the energy quota for the app as possible, it is important to dispose of the session when you are done with your work for the session so that the app can move into the **Suspended** state more quickly.

Disposing of the session yourself, rather than waiting for the revocation event, reduces your app's energy quota usage. This means that your app will be permitted to run in the background longer in future sessions because you'll have more energy quota available to do so. You must maintain a reference to the **ExtendedExecutionSession** object until the end of the operation so that you can call its **Dispose** method.

A snippet that disposes an extended execution session follows:

CS

```
void ClearExtendedExecution(ExtendedExecutionSession session)
{
    if (session != null)
    {
        session.Revoked -= SessionRevoked;
        session.Dispose();
        session = null;
    }
}
```

[See code sample](#)

An app can only have one **ExtendedExecutionSession** active at a time. Many apps use asynchronous tasks in order to complete complex operations that require access to resources such as storage, network, or network-based services. If an operation requires multiple asynchronous tasks to complete, then the state of each of these tasks must be accounted for before disposing the **ExtendedExecutionSession** and allowing the app to be suspended. This requires reference counting the number of tasks that are still running, and not disposing of the session until that value reaches zero.

Here is some example code for managing multiple tasks during an extended execution session period. For more information on how to use this in your app please see the code sample linked below:

```

static class ExtendedExecutionHelper
{
    private static ExtendedExecutionSession session = null;
    private static int taskCount = 0;

    public static bool IsRunning
    {
        get
        {
            if (session != null)
            {
                return true;
            }
            else
            {
                return false;
            }
        }
    }

    public static async Task<ExtendedExecutionResult>
    RequestSessionAsync(ExtendedExecutionReason reason,
        TypedEventHandler<object, ExtendedExecutionRevokedEventArgs> revoked, String
        description)
    {
        // The previous Extended Execution must be closed before a new one
        can be requested.
        ClearSession();

        var newSession = new ExtendedExecutionSession();
        newSession.Reason = reason;
        newSession.Description = description;
        newSession.Revoked += SessionRevoked;

        // Add a revoked handler provided by the app in order to clean up an
        operation that had to be halted prematurely
        if(revoked != null)
        {
            newSession.Revoked += revoked;
        }

        ExtendedExecutionResult result = await
        newSession.RequestExtensionAsync();

        switch (result)
        {
            case ExtendedExecutionResult.Allowed:
                session = newSession;
                break;
            default:
            case ExtendedExecutionResult.Denied:
                newSession.Dispose();
        }
    }
}

```

```

        break;
    }
    return result;
}

public static void ClearSession()
{
    if (session != null)
    {
        session.Dispose();
        session = null;
    }

    taskCount = 0;
}

public static Deferral GetExecutionDeferral()
{
    if (session == null)
    {
        throw new InvalidOperationException("No extended execution
session is active");
    }

    taskCount++;
    return new Deferral(OnTaskCompleted);
}

private static void OnTaskCompleted()
{
    if (taskCount > 0)
    {
        taskCount--;
    }

    //If there are no more running tasks than end the extended lifetime
    by clearing the session
    if (taskCount == 0 && session != null)
    {
        ClearSession();
    }
}

private static void SessionRevoked(object sender,
ExtendedExecutionRevokedEventArgs args)
{
    //The session has been prematurely revoked due to system
    constraints, ensure the session is disposed
    if (session != null)
    {
        session.Dispose();
        session = null;
    }

    taskCount = 0;
}

```

```
}  
}
```

[See code sample](#) 

Ensure that your app uses resources well

Tuning your app's memory and energy use is key to ensuring that the operating system will allow your app to continue to run when it is no longer the foreground app. Use the [Memory Management APIs](#) to see how much memory your app is using. The more memory your app uses, the harder it is for the OS to keep your app running when another app is in the foreground. The user is ultimately in control of all background activity that your app can perform and has visibility on the impact your app has on battery use.

Use [BackgroundExecutionManager.RequestAccessAsync](#) to determine if the user has decided that your app's background activity should be limited. Be aware of your battery usage and only run in the background when it is necessary to complete an action that the user wants.

See also

[Extended Execution Sample](#) 

[Application Lifecycle](#)

[App Lifecycle - Keep Apps Alive with Background Tasks and Extended Execution](#)

[Background Memory Management](#)

[Background Transfers](#)

[Battery Awareness and Background Activity](#) 

[MemoryManager class](#)

[Play Media in the Background](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Tiles for Windows apps

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

A *tile* is an app's representation on the Start menu. Every app has a tile. When you create a new Windows app project in Microsoft Visual Studio, it includes a default tile that displays your app's name and logo. Windows displays this tile when your app is first installed. After your app is installed, you can change your tile's content through notifications; for example, you can change the tile to communicate new information to the user, such as news headlines, or the subject of the most recent unread message.

Configure the default tile

When you create a new project in Visual Studio, it creates a simple default tile that displays your app's name and logo.

To edit your tile, double click the **Package.appxmanifest** file in your main UWP project to open the designer (or right click the file and select View Code).

XML

```
<Applications>
  <Application Id="App"
    Executable="$targetnametoken$.exe"
    EntryPoint="ExampleApp.App">
    <uap:VisualElements
      DisplayName="ExampleApp"
      Square150x150Logo="Assets\Square150x150Logo.png"
      Square44x44Logo="Assets\Square44x44Logo.png"
      Description="ExampleApp"
      BackgroundColor="#464646">
      <uap:SplashScreen Image="Assets\SplashScreen.png" />
    </uap:VisualElements>
  </Application>
</Applications>
```

There are a few items you should update:

- **DisplayName:** Replace this value with the name you want to display on your tile.
- **ShortName:** Because there is limited room for your display name to fit on tiles, we recommend that you to specify a ShortName as well, to make sure your app's name doesn't get truncated.
- **Logo images:**

You should replace these images with your own. You have the option of supplying images for different visual scales, but you are not required to supply them all. To ensure that your app looks good on a range of devices, we recommend that you provide 100%, 200%, and 400% scale versions of each image. See [Construct your Windows app's icon](#) to learn more about generating these assets.

Scaled images follow this naming convention:

<image name>.scale-<scale factor>.<image file extension>

For example: SplashScreen.scale-100.png

When you refer to the image, you refer to it as *<image name>.<image file extension>* ("SplashScreen.png" in this example). The system will automatically select the appropriate scaled image for the device from the images you've provided.

- You don't have to, but we highly recommend supplying logos for wide and large tile sizes so that the user can resize your app's tile to those sizes. To provide these additional images, you create a **DefaultTile** element and use the **Wide310x150Logo** and **Square310x310Logo** attributes to specify the additional images:

XML

```
<Applications>
  <Application Id="App"
    Executable="$targetnametoken$.exe"
    EntryPoint="ExampleApp.App">
    <uap:VisualElements
      DisplayName="ExampleApp"
      Square150x150Logo="Assets\Square150x150Logo.png"
      Square44x44Logo="Assets\Square44x44Logo.png"
      Description="ExampleApp"
      BackgroundColor="#464646">
      <uap:DefaultTile
        Wide310x150Logo="Assets\Wide310x150Logo.png"
        Square310x310Logo="Assets\Square310x310Logo.png">
      </uap:DefaultTile>
      <uap:SplashScreen Image="Assets\SplashScreen.png" />
    </uap:VisualElements>
  </Application>
</Applications>
```

```
</uap:VisualElements>  
</Application>  
</Applications>
```

Use notifications to customize your tile

After your app is installed, you can use notifications to customize your tile. You can do this the first time your app launches or in response to an event, such as a push notification.

To learn how to send tile notifications, see [Send a local tile notification](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

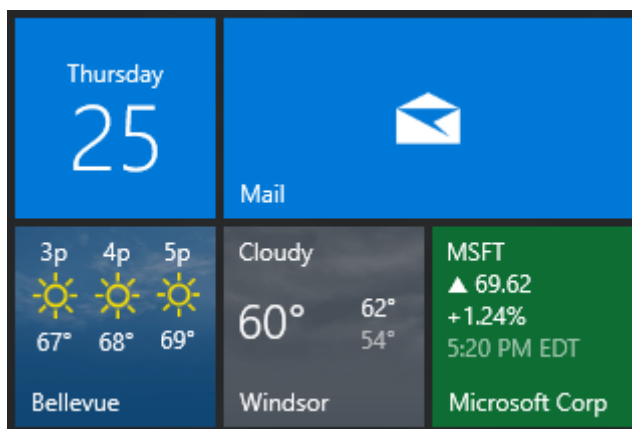
Secondary tiles

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Secondary tiles allow users to pin specific content and deep links from your app onto their Start menu, providing easy future access to the content within your app.



For example, users can pin the weather for numerous specific locations on their Start menu, which provides (1) easy live glanceable information about the current weather thanks to Live Tiles, and (2) a quick entry point to the specific city's weather they care about. Users can also pin specific stocks, news articles, and more items that are important to them.

By adding secondary tiles to your app, you help the user re-engage quickly and efficiently with your app, encouraging them to return more often thanks to the easy access that secondary tiles provides.

Only users can pin a secondary tile; apps cannot pin secondary tiles programmatically without user approval. The user must explicitly click a "Pin" button within your app, at which point you then use the API to request to create a secondary tile, and then the system displays a dialog box asking the user to confirm whether they would like the tile pinned.

Quick links

Article	Description
Guidance on secondary tiles	Learn about when and where you should use secondary tiles.
Pin secondary tiles	Learn how to pin a secondary tile.
Pin from desktop apps	Desktop apps can pin secondary tiles thanks to the Desktop Bridge!

Secondary tiles in relation to primary tiles

Secondary tiles are associated with a single parent app. They are pinned to the Start menu to provide a user with a consistent and efficient way to launch directly into a frequently used area of the parent app. This can be either a general subsection of the parent app that contains frequently updated content, or a deep link to a specific area in the app.

Examples of secondary tile scenarios include:

- Weather updates for a specific city in a weather app
- A summary of upcoming events in a calendar app
- Status and updates from an important contact in a social app
- Specific feeds in an RSS reader
- A music playlist
- A blog

Any frequently changing content that a user wants to monitor is a good candidate for a secondary tile. After the secondary tile is pinned, users can receive at-a-glance updates through the tile and use it to launch directly into the parent app.

Secondary tiles are similar to primary tiles in many ways:

- They use tile notifications to display rich content.
- They must include a 150 x 150 pixel logo for the default tile content.
- They can optionally include the other logo sizes to enable larger tile sizes.
- They can show notifications and badges.
- They can be rearranged on the Start menu.
- They are automatically deleted when the app is uninstalled.
- Their badge and lock detailed status text can be shown on lock.

However, secondary tiles differ from primary tiles in some noticeable ways:

- Users can delete their secondary tiles at any time without deleting the parent app.

- Secondary tiles can be created at run time. App tiles can be created only during installation.
- A flyout prompts the user for confirmation before adding a secondary tile.
- They cannot be programmatically selected for the lock screen through a request to the user. The user must manually add the secondary tile through the Personalize page in PC Settings.

For sending notifications, specific methods are provided for tile and badge updaters and push notification channels used with secondary tiles. These parallel the versions used with primary tiles. For instance, `CreateBadgeUpdaterForApplication` vs. `CreateBadgeUpdaterForSecondaryTile`.

Guidance on secondary tiles

To learn about when and where you should use secondary tiles, and other usage guidance, please see [Guidance for secondary tiles](#)

Pinning secondary tiles

To learn how to pin secondary tiles, please see [Pin secondary tiles](#).

Desktop applications and secondary tiles

To learn how to use secondary tiles from your desktop application via the Desktop Bridge, please see [Pin secondary tiles from desktop apps](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Secondary tile guidance

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

A secondary tile provides a consistent, efficient way for users to directly access specific areas within an app from the Start menu. Although a user chooses whether or not to "pin" a secondary tile to the Start menu, the pinnable areas in an app are determined by the developer. For a more detailed summary, see [Secondary tiles overview](#). Consider these guidelines when you enable secondary tiles and design the associated UI in your app.

ⓘ Note

Only users can pin a secondary tile to the Start menu; apps can't programmatically pin secondary tiles. Users also control tile removal, and can remove a secondary tile from the Start menu or from within the parent app.

Recommendations

Consider the following recommendations when enabling secondary tiles in your app:

- When the content in focus is pinnable, the app bar should contain a "Pin to Start" button to create a secondary tile for the user.
- When the user clicks "Pin to Start", you should immediately call the API from the UI thread to [pin the secondary tile](#).
- If the content in focus is already pinned, replace the "Pin to Start" button on the app bar with an "Unpin from Start" button. The "Unpin from Start" button should remove the existing secondary tile.
- When the content in focus is not pinnable, don't show a "Pin to Start" button (or show a disabled "Pin to Start" button).
- Use the system-provided glyphs for your "Pin to Start" and "Unpin from Start" buttons (see the pin and unpin members in `Windows.UI.Xaml.Controls.Symbol` or `WinJS.UI.AppBarIcon`).

- Use the standard button text: "Pin to Start" and "Unpin from Start". You'll have to override the default text when using the system-provided pin and unpin glyphs.
- Don't use a secondary tile as a virtual command button to interact with the parent app, such as a "skip to next track" tile.

Additional usage guidance for devs

- When an app launches, it should always enumerate its secondary tiles, in case there were any additions or deletions of which it was unaware. When a secondary tile is deleted through the Start screen app bar, Windows simply removes the tile. The app itself is responsible for releasing any resources that were used by the secondary tile. When secondary tiles are copied through the cloud, current tile or badge notifications on the secondary tile, scheduled notifications, push notification channels, and Uniform Resource Identifiers (URIs) used with periodic notifications are not copied with the secondary tile and must be set up again.
- An app should use meaningful, re-creatable, unique IDs for secondary tiles. Using predictable secondary tile IDs that are meaningful to an app helps the app understand what to do with these tiles when they are seen in a fresh installation on a new computer.
 - At runtime, the app can query whether a specific tile exists.
 - The secondary tile platform can be asked to return the set of all secondary tiles belonging to a specific app. Using meaningful, unique IDs for these tiles can help the app examine the set of secondary tiles and perform appropriate actions. For instance, for a social media app, IDs could identify individual contacts for whom tiles were created.
- Secondary tiles, like all tiles on the Start screen, are dynamic outlets that can be frequently updated with new content. Secondary tiles can surface notifications and updates by using the same mechanisms as any other tile. See [Choose a notification delivery method](#) to learn more.

Related

- [Secondary tiles overview](#)
 - [Pin secondary tiles](#)
 - [Tile assets](#)
 - [Tile content documentation](#)
 - [Send a local tile notification](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

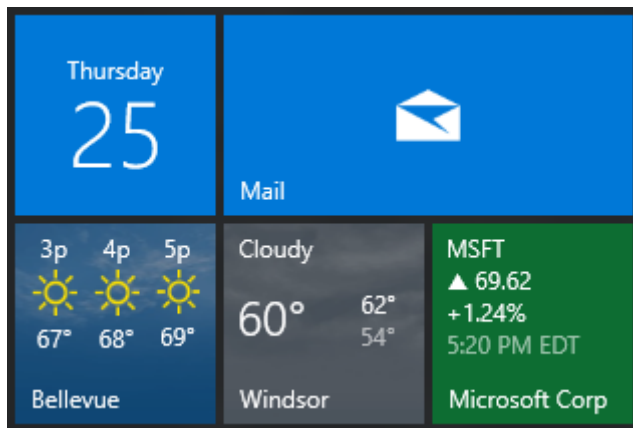
Pin secondary tiles to Start

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

This topic walks you through the steps to create a secondary tile for your Windows app and pin it to the Start menu.



To learn more about secondary tiles, please see the [Secondary tiles overview](#).

Add namespace

The `Windows.UI.StartScreen` namespace includes the `SecondaryTile` class.

C#

```
using Windows.UI.StartScreen;
```

Initialize the secondary tile

Secondary tiles are composed of a few key components...

- **TileId:** A unique identifier that lets you identify the tile among your other secondary tiles.
- **DisplayName:** The name you want shown on the tile.

- **Arguments:** The arguments you want passed back to your app when the user clicks your tile.
- **Square150x150Logo:** The required logo, displayed on the medium size tile (and resized to small size tile if no small logo provided).

You **MUST** provide initialized values for all of the above properties, or else you will get an exception.

There are a variety of constructors you can use, but using the constructor that takes in the `tileId`, `displayName`, `arguments`, `square150x150Logo`, and `desiredSize` helps ensure you set all of the required properties.

C#

```
// Construct a unique tile ID, which you will need to use later for updating
the tile
string tileId = "City" + zipCode;

// Use a display name you like
string displayName = cityName;

// Provide all the required info in arguments so that when user
// clicks your tile, you can navigate them to the correct content
string arguments = "action=viewCity&zipCode=" + zipCode;

// Initialize the tile with required arguments
SecondaryTile tile = new SecondaryTile(
    tileId,
    displayName,
    arguments,
    new Uri("ms-appx:///Assets/CityTiles/Square150x150Logo.png"),
    TileSize.Default);
```

Optional: Add support for larger tile sizes

If you're going to be displaying rich tile notifications on your secondary tile, you'll likely want to allow the user to resize their tile to be wide or large, so that they can see even more of your content.

To enable wide and large tile sizes, you need to provide the *Wide310x150Logo* and *Square310x310Logo*. Also, if possible, you should provide the *Square71x71Logo* for the small tile size (otherwise we will downsize your required *Square150x150Logo* for the small tile).

You can also provide a unique *Square44x44Logo*, which is optionally displayed on the bottom right corner when a notification is present. If you don't provide one, the

Square44x44Logo from your primary tile will be used instead.

C#

```
// Enable wide and large tile sizes
tile.VisualElements.Wide310x150Logo = new Uri("ms-appx:///Assets/CityTiles/Wide310x150Logo.png");
tile.VisualElements.Square310x310Logo = new Uri("ms-appx:///Assets/CityTiles/Square310x310Logo.png");

// Add a small size logo for better looking small tile
tile.VisualElements.Square71x71Logo = new Uri("ms-appx:///Assets/CityTiles/Square71x71Logo.png");

// Add a unique corner logo for the secondary tile
tile.VisualElements.Square44x44Logo = new Uri("ms-appx:///Assets/CityTiles/Square44x44Logo.png");
```

Optional: Enable showing the display name

By default the display name will NOT be shown. To show the display name on medium/wide/large, add the following code.

C#

```
// Show the display name on all sizes
tile.VisualElements.ShowNameOnSquare150x150Logo = true;
tile.VisualElements.ShowNameOnWide310x150Logo = true;
tile.VisualElements.ShowNameOnSquare310x310Logo = true;
```

Optional: 3D secondary tiles

You can enhance your secondary tile for Windows Mixed Reality by adding 3D assets. Users can place 3D tiles directly in their Windows Mixed Reality home instead of the Start menu when using your app in a Mixed Reality environment. For example, you can create 360° photospheres that link directly into a 360° photo viewer app, or let users place a 3D model of a chair from a furniture catalog that opens a details page about the pricing and color options for that object when selected. To get started, refer to the [Mixed Reality developer documentation](#).

Pin the secondary tile

Finally, request to pin the tile. Note that this must be called from a UI thread. On Desktop, a dialog will appear asking the user to confirm whether they would like to pin the tile.

Important

For desktop applications that use the Desktop Bridge, you must first perform an extra step as described in [Pin from desktop apps](#)

C#

```
// Pin the tile
bool isPinned = await tile.RequestCreateAsync();

// TODO: Update UI to reflect whether user can now either unpin or pin
```

Check if a secondary tile exists

If your user visits a page in your app that they have already pinned to Start, you will want to instead display an "Unpin" button.

Therefore, when choosing what button to display, you need to first check whether the secondary tile is currently pinned.

C#

```
// Check if the secondary tile is pinned
bool isPinned = SecondaryTile.Exists(tileId);

// TODO: Update UI to reflect whether user can either unpin or pin
```

Unpinning a secondary tile

If the tile is currently pinned, and the user clicks your unpin button, you'll want to unpin (delete) the tile.

C#

```
// Initialize a secondary tile with the same tile ID you want removed
SecondaryTile toBeDeleted = new SecondaryTile(tileId);
```

```
// And then unpin the tile
await toBeDeleted.RequestDeleteAsync();
```

Updating a secondary tile

If you need to update the logos, display name, or anything else on the secondary tile, you can use `RequestUpdateAsync`.

C#

```
// Initialize a secondary tile with the same tile ID you want to update
SecondaryTile tile = new SecondaryTile(tileId);

// Assign ALL properties, including ones you aren't changing

// And then update it
await tile.UpdateAsync();
```

Enumerating all pinned secondary tiles

If you need to discover all the tiles your user has pinned, instead of using `SecondaryTile.Exists`, you can alternatively use `SecondaryTile.FindAllAsync`.

C#

```
// Get all secondary tiles
var tiles = await SecondaryTile.FindAllAsync();
```

Send a tile notification

To learn how to display rich content on your tile via tile notifications, please see [Send a local tile notification](#).

Related

- [Secondary tiles overview](#)
 - [Secondary tiles guidance](#)
 - [Tile content documentation](#)
 - [Send a local tile notification](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

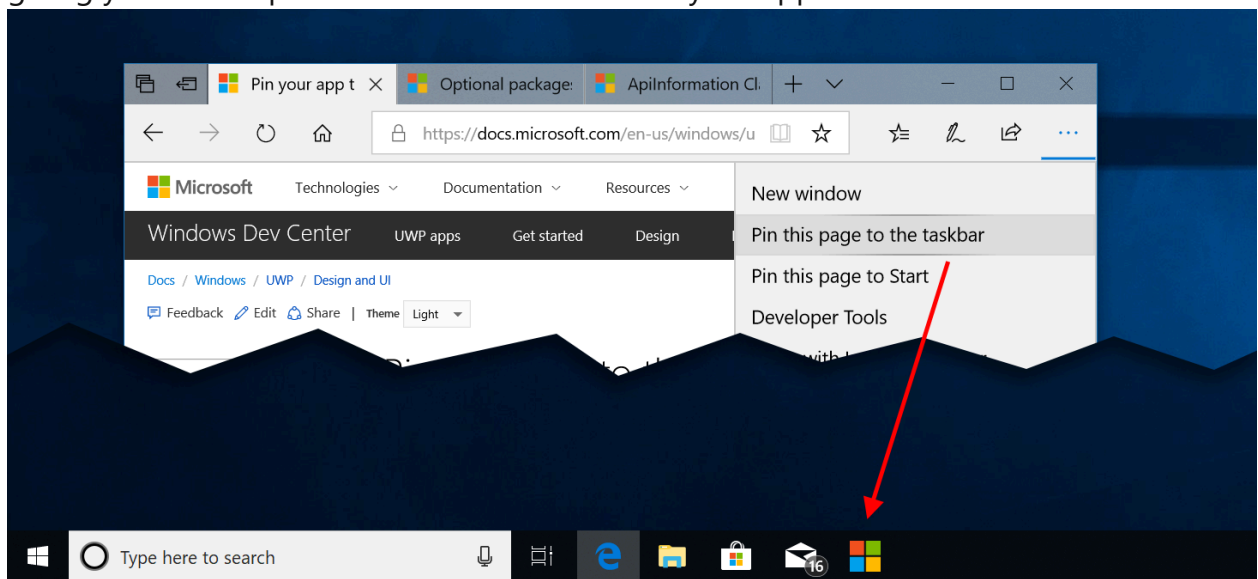
Pin secondary tiles to taskbar

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Just like pinning secondary tiles to Start, you can pin secondary tiles to the taskbar, giving your users quick access to content within your app.



ⓘ Important

Limited Access API: This API is a limited access feature. To use this API, please contact taskbarsecondarytile@microsoft.com.

Requires October 2018 Update: You must target SDK 17763 and be running build 17763 or later to pin to taskbar.

Guidance

A secondary tile provides a consistent, efficient way for users to directly access specific areas within an app. Although a user chooses whether or not to "pin" a secondary tile to the taskbar, the pinnable areas in an app are determined by the developer. For more guidance, see [Secondary tile guidance](#).

1. Determine if API exists and unlock Limited-Access

Older devices don't have the taskbar pinning APIs (if you're targeting older versions of Windows 10). Therefore, you shouldn't display a pin button on these devices that aren't capable of pinning.

Additionally, this feature is locked under Limited-Access. To gain access, contact Microsoft. API calls to [TaskbarManager.RequestPinSecondaryTileAsync](#), [TaskbarManager.IsSecondaryTilePinnedAsync](#), and [TaskbarManager.TryUnpinSecondaryTileAsync](#) will fail with an Access Denied exception. Apps are not allowed to use this API without permission, and the API definition may change at any time.

Use the [ApiInformation.IsMethodPresent](#) method to determine if the APIs are present. And then use the [LimitedAccessFeatures](#) API to try unlocking the API.

C#

```
if (ApiInformation.IsMethodPresent("Windows.UI.Shell.TaskbarManager",
    "RequestPinSecondaryTileAsync"))
{
    // API present!
    // Unlock the pin to taskbar feature
    var result = LimitedAccessFeatures.TryUnlockFeature(
        "com.microsoft.windows.secondarytilemanagement",
        "<tokenFromMicrosoft>",
        "<publisher> has registered their use of
com.microsoft.windows.secondarytilemanagement with Microsoft and agrees to
the terms of use.");

    // If unlock succeeded
    if ((result.Status == LimitedAccessFeatureStatus.Available) ||
        (result.Status == LimitedAccessFeatureStatus.AvailableWithoutToken))
    {
        // Continue
    }
    else
    {
        // Don't show pin to taskbar button or call any of the below APIs
    }
}

else
{
    // Don't show pin to taskbar button or call any of the below APIs
}
```

2. Get the TaskbarManager instance

Windows apps can run on a wide variety of devices; not all of them support the taskbar. Right now, only Desktop devices support the taskbar. Additionally, presence of the taskbar might come and go. To check whether taskbar is currently present, call the [TaskbarManager.GetDefault](#) method and check that the instance returned is not null. Don't display a pin button if the taskbar isn't present.

We recommend holding onto the instance for the duration of a single operation, like pinning, and then grabbing a new instance the next time you need to do another operation.

C#

```
TaskbarManager taskbarManager = TaskbarManager.GetDefault();

if (taskbarManager != null)
{
    // Continue
}
else
{
    // Taskbar not present, don't display a pin button
}
```

3. Check whether your tile is currently pinned to the taskbar

If your tile is already pinned, you should display an unpin button instead. You can use the [IsSecondaryTilePinnedAsync](#) method to check whether your tile is currently pinned (users can unpin it at any time). In this method, you pass the `TileId` of the tile you want to know is pinned.

C#

```
if (await taskbarManager.IsSecondaryTilePinnedAsync("myTileId"))
{
    // The tile is already pinned. Display the unpin button.
}

else
{
    // The tile is not pinned. Display the pin button.
}
```